

# Verifying the Noble Invoice Function in Azure

A beginner-friendly, command-by-command walkthrough with expected output at every step

Prepared for George Tsang · Noble & Associates Property Management · June 23, 2026

**What this guide is for.** You deployed the Noble invoice Function to Azure. A deploy only confirms your code *uploaded* — not that it *works*. This guide walks you through four checks (called “layers”) that confirm the Function actually runs in Azure and can reach the two services it depends on: Content Understanding (which reads the invoice) and Table Storage (the ledger, which records what happened). Follow it top to bottom. You do not need to understand Azure deeply — just run each command and compare what you see to the “What you should see” boxes.

## Two rules before you start

**Rule 1 — stay in ONE PowerShell window.** Throughout this guide you will save values into *variables* (things like `$APP` and `$KEY`). These variables only exist inside the PowerShell window you are using. If you close that window, the variables are gone and you must start again from Part 2. So: open one PowerShell window now and keep it open until you finish.

**Rule 2 — when you copy a command, copy the whole line.** Each command is a complete instruction. Paste it, press Enter, read the result, then move on.

**How to open PowerShell:** press the Windows key, type `PowerShell`, and click **Windows PowerShell**. You’ll see a prompt like `PS C:\Users\georg>`. That’s where every command below goes.

**A quick note on variables.** When you run a line like `$APP = "something"`, PowerShell stores `something` under the name `$APP` and prints nothing — that’s normal. To check what a variable holds, just type its name and press Enter:

```
$APP
```

If it prints your value, it’s set. If it prints nothing, it’s empty and you need to set it again.

## Part 1 — One-time setup

You only do this part once per machine.

### Step 1.1 — Confirm you are signed in

**What this does:** checks that `az` (the Azure command-line tool) knows who you are.

```
az account show
```

**What you should see:** a block of JSON with your account details, including a `"name"` (your subscription name) and `"user"`. If instead you see an error about not being logged in, run `az login` and complete the sign-in, then try again.

### Step 1.2 — Install the Resource Graph add-on

**What this does:** adds a search tool to `az` that can find your Function App across every subscription you have access to. You need this in Part 2.

```
az extension add --name resource-graph --only-show-errors
```

**What you should see:** nothing, or a short note. Running it twice is harmless. If it prints nothing and returns you to the prompt, it worked.

### Step 1.3 — Check for Azure Functions Core Tools (optional)

**What this does:** checks whether the `func` command is installed. It’s used for one convenient command later, but there is an alternative if you don’t have it, so don’t worry if this fails.

```
func --version
```

**What you should see:** a version starting with `4` (for example `4.0.6280`). If you get an error that `func` is not recognized, that’s fine — this guide gives you an `az` alternative wherever `func` is used.

## Part 2 — Find your Function App and point az at it

When you deployed, the tooling created your Function App inside *some* resource group and *some* subscription. Before you can test it, you need to find exactly where it lives and tell `az` to focus there.

### Step 2.1 — Search for the app

**What this does:** searches all your subscriptions for a web/function app whose name contains `invoiceprocess`.

```
az graph query -q "Resources | where type =~ 'microsoft.web/sites' | where name contains 'invoiceprocess' | project name, resourceGroup, subscriptionId, location, kind" -o table
```

**What you should see:** a small table, something like this (your exact values will differ):

| Name                                        | ResourceGroup         | SubscriptionId                       | Location | Kind              |
|---------------------------------------------|-----------------------|--------------------------------------|----------|-------------------|
| func-invoiceprocess-westus-exg8hzhke9c9ajgs | rg-invoiceprocess-dev | 1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d | westus   | functionapp,linux |

Three values from this table matter: **Name**, **ResourceGroup**, and **SubscriptionId**. Keep this table on screen for the next two steps.

**If the table is empty:** either you're signed in to the wrong Azure account/tenant, or the app genuinely isn't deployed. Double-check with `az account show` that the subscription name looks right. If it's the wrong tenant, run `az login --tenant <your-tenant-id>` and retry.

### Step 2.2 — Point az at the right subscription

**What this does:** tells `az` that all following commands should target the subscription your app is in. Copy the `SubscriptionId` from the table above into the quotes.

```
az account set --subscription "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d"
```

**Confirm it worked:**

```
az account show -o table
```

**What you should see:** a one-line table showing that same subscription as the active one.

### Step 2.3 — Save the app name and resource group into variables

**What this does:** stores the Name and ResourceGroup so you don't have to retype them. Copy your real values from the Step 2.1 table into the quotes.

```
$APP = "func-invoiceprocess-westus-exg8hzhke9c9ajgs"
$RG = "rg-invoiceprocess-dev"
```

**Confirm they're set** (type each name and press Enter):

```
$APP
$RG
```

Each should print the value you just entered.

### Step 2.4 — Confirm the app is running and save its web address

**What this does:** checks the app's status and saves its public web address (hostname) into `$APPHOST`.

```
az functionapp show -g $RG -n $APP --query "{state:state, kind:kind, host:defaultHostName}" -o jsonc
```

**What you should see:**

```
{
  "host": "func-invoiceprocess-westus-exg8hzhke9c9ajgs.azurewebsites.net",
  "kind": "functionapp,linux",
  "state": "Running"
}
```

The important word is `"state": "Running"`. Now save the host:

```
$APPHOST = (az functionapp show -g $RG -n $APP --query defaultHostName -o tsv)
$APPHOST
```

We use the name `$APPHOST` on purpose. Do **not** use `$HOST` — PowerShell already uses that name internally, and overwriting it can cause confusing errors.

### Part 3 — Pre-flight: the two things that usually break a deployment

For a brand-new deployment, two problems cause almost every failure. Checking them now means the tests in Part 6 give clean results instead of confusing errors.

- **Your settings didn't come along.** When you deploy, your code uploads but your local config file ( `local.settings.json` ) stays on your laptop. The Function in Azure only has the settings you explicitly set on it. If a setting is missing, the Function starts but fails the moment it runs.
- **The app can't talk to other services yet.** The Function reaches Content Understanding and Table Storage using a *managed identity* — think of it as an ID badge the app carries so it can open doors to other Azure services without a password. That badge needs the right permissions granted to it.

#### Step 3.1 — Check the app's settings

**What this does:** lists the names of all settings currently on the Function App.

```
az functionapp config appsettings list -g $RG -n $APP --query "[].name" -o tsv
```

**What you should see:** a list of names. Check that every one of these is present:

| Setting name                              | What it should be                                                           | Why it matters                        |
|-------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------|
| <code>AZURE_CU_ENDPOINT</code>            | <code>https://invoice-processing-dev-resource.services.ai.azure.com/</code> | where to send the invoice for reading |
| <code>AZURE_CU_KEY</code>                 | your Content Understanding key                                              | password for that service             |
| <code>AZURE_CU_ANALYZER_ID</code>         | <code>invoicerouter</code>                                                  | the router model                      |
| <code>AZURE_CU_GENERAL_ANALYZER_ID</code> | <code>generalinvoice</code>                                                 | the extraction model                  |
| <code>AZURE_CU_API_VERSION</code>         | <code>2025-11-01</code>                                                     | service version                       |
| <code>AZURE_STORAGE_ACCOUNT</code>        | <code>stinvoicedevwestus</code>                                             | where the ledger lives                |
| <code>AZURE_TABLES_TABLE</code>           | <code>InvoiceExtractProcessLog</code>                                       | the ledger table                      |
| <code>AZURE_CLIENT_ID</code>              | <code>086c04ae-e652-4c25-bee5-43ce0bd566cd</code>                           | tells the app which ID badge to use   |
| <code>FIELD_CONFIDENCE_THRESHOLD</code>   | <code>0.75</code>                                                           | a tuning value for the gates          |

**If one is missing or wrong,** set it like this (this example sets `AZURE_CLIENT_ID` , the one most often forgotten — replace the name and value as needed):

```
az functionapp config appsettings set -g $RG -n $APP --settings AZURE_CLIENT_ID=086c04ae-e652-4c25-bee5-43ce0bd566cd
```

Changing a setting restarts the app. Wait about 30–60 seconds before running the tests in Part 6, or the first test may hit the app mid-restart. Also: if the `AZURE_CU_KEY` value here is the same one that was shared in your local config file, regenerate it in the portal (the resource's **Keys and Endpoint** page) and update it here — a stale key is a common cause of the `502` error later.

#### Step 3.2 — Check the app's ID badge and its permissions

**What this does:** confirms the managed identity ( `id-invoice-processing-dev` ) is attached to the app.

```
az functionapp identity show -g $RG -n $APP -o jsonc
```

**What you should see:** a block that includes `userAssignedIdentities` with an entry mentioning `id-invoice-processing-dev` . (An empty result means no identity is attached — the ledger writes in Layer 4 will fail until one is.)

**Now check that badge has permission to write to the ledger:**

```
az role assignment list --assignee 086c04ae-e652-4c25-bee5-43ce0bd566cd --all -o table
```

**What you should see:** at least one row showing the role **Storage Table Data Contributor** scoped to `stinvoicedevwestus` . That role is what lets the Function write to the ledger. (If you ever switch Content Understanding to use the identity instead of a key, you'd also need **Cognitive Services User** here — but with a key set, you don't need it today.)

## Part 4 — Get the function key

In Azure, the Function won't run for just anyone — it requires a key, similar to a password, on every call. You'll get that key now and save it.

### Step 4.1 — List the function and its key

**What this does:** shows the Function's web address with the key already attached.

```
func azure functionapp list-functions $APP --show-keys
```

**What you should see:**

```
Functions in func-invoiceprocess-westus-exg8zhke9c9ajgs:
  process_invoice - [httpTrigger]
    Invoke url: https://func-invoiceprocess-westus-...azurewebsites.net/api/process-invoice?code=AbC123...==
```

Two things to notice: the **function name** is `process_invoice` (you'll need that exact name in the next step), and the **Invoke url** ends with `?code=...` — that long string after `code=` is the key.

If **func** isn't installed (Step 1.3 failed), use this `az` alternative instead to confirm the function exists:

```
az functionapp function list -g $RG -n $APP -o table
```

### Step 4.2 — Save the key into a variable

**What this does:** fetches just the key and stores it in `$KEY`.

```
$KEY = (az functionapp function keys list -g $RG -n $APP --function-name process_invoice --query default -o tsv)
```

**Confirm it's set** (this should print a long string, not an empty line):

```
$KEY
```

If `$KEY` prints nothing, double-check the function name in Step 4.1 and use that exact name in the command above.

## Part 5 — Save the test script

The tests use a small Python script called `verify_fn.py`. It takes an invoice PDF, sends it to your Function, and prints the result in a readable way.

### Step 5.1 — Put the script in your scripts folder

You have two options.

**Option A (easiest):** use the `verify_fn.py` file provided alongside this PDF. Move it into your scripts folder: `C:\Users\georg\dev\Noble Homes\Invoice Extractor\noble-invoice-process\scripts\`

**Option B (type it yourself):** open VS Code (or Notepad), paste the full script from the **Appendix** at the end of this document, and save it as `verify_fn.py` in that same `scripts` folder. Make sure the file is named exactly `verify_fn.py` (not `verify_fn.py.txt`).

### Step 5.2 — Move into the scripts folder

**What this does:** changes your PowerShell location to the scripts folder so you can run the script. (Your variables `$APP`, `$KEY`, etc. stay set — changing folder does not clear them.)

```
cd "C:\Users\georg\dev\Noble Homes\Invoice Extractor\noble-invoice-process\scripts"
```

You'll also need an invoice PDF to test with. If you have a `samples` folder with test invoices, you can point at one of those. Anywhere this guide writes `..\samples\invoice1.pdf`, replace it with the path to a real invoice PDF you have.

## Part 6 — The four-layer check

This is the actual verification. Each layer proves more than the one before:

1. **Host registration** — the Function loaded correctly in Azure.
2. **Bad input → rejected** — your code is running and checking its inputs.
3. **Real invoice → processed** — reading, gates, and ledger all work in Azure.
4. **Downstream connections** — the app reached Content Understanding and the ledger.

### Layer 1 — Is the Function registered?

You already saw this in Step 4.1: running `func azure functionapp list-functions` showed `process_invoice` in the list. That listing **is** Layer 1 passing — it proves the deployment didn't just upload, it actually loaded the Function.

**What “good” looks like:** `process_invoice` appears in the list.

**If the list was empty:** the package uploaded but failed to load — almost always a missing setting (go back to Part 3.1) or a packaging problem. After fixing settings, redeploy from the `functionapp` folder with `func azure functionapp publish $APP`, then re-check.

### Layer 2 — Does the Function reject bad input?

**What this does:** sends an empty request on purpose. A correctly-running Function should refuse it.

```
python verify_fn.py --base-url "https://$APPHOST" --key $KEY --bad-payload --insecure
```

**What you should see:**

```
HTTP 400
{
  "error": "sourceId is required (use the SharePoint item UniqueId)"
}
```

Here, **HTTP 400** is a **PASS**. It means your request reached the Function and the Function's own code rejected the empty input — exactly what it should do.

**If you see HTTP 401:** the key is wrong or missing, not a code problem. Re-run Step 4.2 to refresh `$KEY`.

**If you see a connection or certificate error:** keep the `--insecure` flag on the command (it tells the script to tolerate your corporate network's security inspection).

### Layer 3 — Does a real invoice get processed?

**What this does:** sends an actual invoice PDF. This is the big test — it exercises reading the invoice, running the decision gates, and writing to the ledger, all in Azure.

```
python verify_fn.py --base-url "https://$APPHOST" --key $KEY --file "..\samples\invoice1.pdf" --insecure
```

**What you should see:** a large block of JSON, ending with two summary lines:

```
--> routingDecision: HAPPY_PATH_CANDIDATE | status: Extracted | alreadyProcessed: False
--> sourceId: 3f2c9a10-7d3e-4a55-9c10-2b8e6f4a1d77
```

**HTTP 200** plus `status: Extracted` means it worked end to end. **Copy the `sourceId` value** (the long code on the second line) — Layer 4 needs it.

The very first call may take several extra seconds. Azure puts idle Functions to sleep to save cost, so the first request “wakes it up.” This is normal — it's not a failure.

**If you see HTTP 502:** the Function couldn't reach Content Understanding — usually a wrong or stale `AZURE_CU_KEY`. Recheck Part 3.1.

**If you see HTTP 500:** a setting or the ledger configuration is missing. Recheck Part 3.1.

### Layer 4 — Did the app reach its downstream services?

The Layer 3 result already contains most of the proof — you just read it:

- **Content Understanding worked** if the JSON has a `fields` section filled with values (vendor name, total, etc.). If reading had failed, you'd have gotten a `502` instead.
- **The ledger worked** if the result shows `status: Extracted` and does **not** contain a line starting with `WARNING ledgerWriteError`. The script prints that warning only if the ledger write failed.

To see the ledger record with your own eyes, run the two commands below. The first saves your `sourceId` and works out the ledger's “partition key” (just the first two characters of the sourceId). The second reads the record.

```
$SID = "3f2c9a10-7d3e-4a55-9c10-2b8e6f4a1d77"
$PK = $SID.Substring(0,2)
az storage entity show --account-name stinvoicedevwestus --table-name InvoiceExtractProcessLog --partition-key $PK --row-key $SID --auth-mode login -o jsonc
```

(Replace the `sourceId` in the first line with the one you copied from Layer 3.)

**What you should see:** a record showing `"Status": "Extracted"` and a `"RoutingDecision"`. That record was written by the app's ID badge — so seeing it confirms the app reached the ledger successfully.

**If you get a 403 error here:** that means *your own* account lacks read permission on the storage — it does **not** mean the test failed. The Layer 3 result (no `ledgerWriteError`) is still valid proof. You can skip this command.

**If the Layer 3 result had a `ledgerWriteError`:** the app's ID badge is missing the **Storage Table Data Contributor** role, or `AZURE_CLIENT_ID` isn't set correctly. Go back to Part 3.1 and 3.2.

## Part 7 — Understanding the result

The `routingDecision` value tells you what the Function decided to do with the invoice. Any of these is a valid, healthy result — they just mean different things:

| routingDecision                  | Plain meaning                                    |
|----------------------------------|--------------------------------------------------|
| HAPPY_PATH_CANDIDATE             | clean invoice, all checks passed                 |
| REVIEW_B3_HANDWRITTEN_OR_UNKNOWN | looks handwritten or unclear — needs a human     |
| REVIEW_B4_CRITICAL_FIELD         | an important field was missing or low-confidence |
| REVIEW_GST_MATH                  | the GST and total don't add up — needs a human   |
| REJECT_B2_OTHER_CATEGORY         | this document isn't an invoice                   |
| REVIEW_NO_CHILD_EXTRACTION       | couldn't extract invoice fields                  |

If you test the same clean invoice twice and it lands on `HAPPY_PATH_CANDIDATE` once and `REVIEW_B4_CRITICAL_FIELD` another time, that's expected — the AI's confidence wobbles slightly between runs, and one field (the vendor name) often sits right at the cutoff. This is a known characteristic, not a deployment bug.

## Part 8 — Troubleshooting at a glance

| What you see                         | What it means                         | What to do                                                                         |
|--------------------------------------|---------------------------------------|------------------------------------------------------------------------------------|
| <code>az</code> says "not logged in" | session expired                       | run <code>az login</code>                                                          |
| Resource Graph table empty           | wrong account/tenant, or not deployed | check <code>az account show</code> ; try <code>az login --tenant &lt;id&gt;</code> |
| <code>\$KEY</code> prints nothing    | wrong function name in Step 4.2       | use the exact name from Step 4.1                                                   |
| <code>HTTP 401</code> on a test      | wrong/missing key                     | re-run Step 4.2                                                                    |
| <code>HTTP 404</code> on a test      | wrong address                         | confirm <code>\$APPHOST</code> ; the path is <code>/api/process-invoice</code>     |
| <code>HTTP 500</code>                | a setting is missing                  | recheck Part 3.1                                                                   |
| <code>HTTP 502</code>                | can't reach Content Understanding     | check/refresh <code>AZURE_CU_KEY</code> (Part 3.1)                                 |
| empty function list (Layer 1)        | deployment didn't load                | fix settings, then <code>func azure functionapp publish \$APP</code>               |
| first test very slow                 | app was asleep (cold start)           | wait and retry — normal                                                            |
| <code>403</code> on the ledger read  | your account lacks storage read       | skip it; the Layer 3 result is enough                                              |
| certificate / connection error       | corporate network inspection          | keep <code>--insecure</code> on the script                                         |

## Part 9 — You're done when...

All four layers pass:

- **Layer 1:** `process_invoice` appears in the function list.
- **Layer 2:** the empty request returns `HTTP 400` .
- **Layer 3:** a real PDF returns `HTTP 200` with `status: Extracted` and a `routingDecision` .
- **Layer 4:** the Layer 3 result has no `ledgerWriteError` , and (optionally) the ledger record shows `Status = Extracted` .

When all four are green, your deployed Function is confirmed working and connected to its services. Keep the verified web address ( `$APPHOST` ) and key ( `$KEY` ) — those are exactly what you'll plug into Power Automate next.

## Quick command reference (in order)

Once you're comfortable, here is the whole sequence in one place. Replace the placeholder values with your own.

```

# 1. setup
az account show
az extension add --name resource-graph --only-show-errors

# 2. find and target the app
az graph query -q "Resources | where type =~ 'microsoft.web/sites' | where name contains 'invoiceprocess' | project name, resourceGroup, subscriptionId, location, kind" -o table
az account set --subscription "<SubscriptionId>"
$APP = "<Name>"
$RG = "<ResourceGroup>"
$APPHOST = (az functionapp show -g $RG -n $APP --query defaultHostName -o tsv)

# 3. pre-flight
az functionapp config appsettings list -g $RG -n $APP --query "[].name" -o tsv
az functionapp identity show -g $RG -n $APP -o jsonc
az role assignment list --assignee 086c04ae-e652-4c25-bee5-43ce0bd566cd --all -o table

# 4. get the key
$KEY = (az functionapp function keys list -g $RG -n $APP --function-name process_invoice --query default -o tsv)

# 5. run the tests
cd "C:\Users\georg\dev\Noble Homes\Invoice Extractor\noble-invoice-process\scripts"
python verify_fn.py --base-url "https://$APPHOST" --key $KEY --bad-payload --insecure
python verify_fn.py --base-url "https://$APPHOST" --key $KEY --file "..\samples\invoice1.pdf" --insecure

# 6. confirm the ledger row (use the sourceId printed above)
$SID = "<sourceId-from-layer-3>"
$PK = $SID.Substring(0,2)
az storage entity show --account-name stinvoicedevwestus --table-name InvoiceExtractProcessLog --partition-key $PK --row-key $SID --auth-mode login -o jsonc

```

## Appendix A — The verification script ( `verify_fn.py` )

Save this as `verify_fn.py` in your `scripts` folder if you didn't use the provided file. It uses only Python's standard library, so there is nothing to install.

```
#!/usr/bin/env python3
"""verify_fn.py - exercise the deployed Noble invoice Function in Azure."""
import argparse, base64, json, ssl, sys, urllib.request, urllib.error, uuid

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--base-url", required=True)
    ap.add_argument("--key", help="function key (sent as x-functions-key header)")
    ap.add_argument("--file", help="path to an invoice PDF")
    ap.add_argument("--source-id", help="override sourceId (default: fresh GUID)")
    ap.add_argument("--reprocess", action="store_true")
    ap.add_argument("--bad-payload", action="store_true")
    ap.add_argument("--insecure", action="store_true")
    args = ap.parse_args()

    url = args.base_url.rstrip("/") + "/api/process-invoice"
    sid = None
    if args.bad_payload:
        body = {}
    else:
        if not args.file:
            ap.error("--file is required unless --bad-payload")
        with open(args.file, "rb") as fh:
            b64 = base64.b64encode(fh.read()).decode("ascii")
            sid = args.source_id or str(uuid.uuid4())
            body = {"sourceId": sid, "contentBase64": b64,
                  "fileName": args.file.replace("\\", "/").split("/")[-1]}
        if args.reprocess:
            body["reprocess"] = True

    req = urllib.request.Request(
        url, data=json.dumps(body).encode("utf-8"), method="POST",
        headers={"Content-Type": "application/json"})
    if args.key:
        req.add_header("x-functions-key", args.key)

    ctx = ssl._create_unverified_context() if args.insecure else None
    try:
        with urllib.request.urlopen(req, context=ctx, timeout=120) as resp:
            status, payload = resp.status, resp.read().decode("utf-8")
    except urllib.error.HTTPError as e:
        status, payload = e.code, e.read().decode("utf-8")
    except Exception as e:
        print("REQUEST FAILED:", e); sys.exit(2)

    print("HTTP", status)
    try:
        obj = json.loads(payload)
        print(json.dumps(obj, indent=2)[:4000])
        if "routingDecision" in obj:
            print("\n--> routingDecision:", obj.get("routingDecision"),
                  "| status:", obj.get("status"),
                  "| alreadyProcessed:", obj.get("alreadyProcessed"))
            if obj.get("ledgerWriteError"):
                print("--> WARNING ledgerWriteError:", obj.get("ledgerWriteError"))
        if sid:
            print("--> sourceId:", sid)
    except ValueError:
        print(payload[:4000])

if __name__ == "__main__":
    main()
```

## Appendix B — If az ever stops working again

If the Azure CLI breaks on your corporate network later, you can run every command in this guide from **Azure Cloud Shell** instead, which runs inside Azure and bypasses your local network entirely: open **portal.azure.com**, click the `>` icon in the top bar, choose **PowerShell**, and run the same commands (drop the `--insecure` flags — they aren't needed there). For Layer 1 in Cloud Shell, use `az functionapp function list -g $RG -n $APP -o table` instead of the `func` command, and use the Cloud Shell upload button to bring in `verify_fn.py` and a sample PDF.